

Finite Automata in Haskell

V. Velkey, W. Vromen

Tuesday 23rd May, 2023

Abstract

We give a toy example of a report in *literate programming* style. The main advantage of this is that source code and documentation can be written and presented next to each other. We use the listings package to typeset Haskell source code nicely.

Contents

1	DFA Implementation	2
2	NFA Implementation	2
3	Implementation of ϵ -NFA-s	3
4	Regular expressions	4
5	Transition functions	5
6	Wrapping it up in an executable	5
7	Simple Tests	6
8	Conclusion	6
	Bibliography	6

1 DFA Implementation

This describes our implementation of Deterministic Finite State Automata. As in the definition, a DFA consists of a tuple $(Q, \Sigma, \delta, q_{start}, A)$ representing the set of states, the alphabet, the transition function, the start state and the accept states.

```
module DFA where

type State = Int
type Symbol = Char
data DFA = DFA { states :: [[State]] -- For ease with subset construct, represent by set of
               states.
               , alphabet :: [Symbol]
               , delta :: Symbol -> [State] -> [State]
               , start :: [State]
               , acceptstate :: [[State]] }
```

We implement a basic evaluation function that upon input from the string, evaluates if the string is in the language.

```
evaluate :: DFA -> String -> Bool
evaluate df st = all ('elem' alphabet df) st && -- captures that all element of string in
    alphabet
    stateArr (start df) st 'elem' acceptstate df where
        stateArr :: [State] -> String -> [State] -- recursively go to state at
            end of string
        stateArr q [] = q
        stateArr q (x:xs) = stateArr (delta df x q) xs
```

We write a toy example DFA on the alphabet $\Sigma = \{0, 1\}$ computing the language of words starting with a 0.

```
zeroStart :: DFA
zeroStart = DFA [[0],[1],[2]] ['0', '1'] deltazero [0] [[1]] where
    deltazero :: Symbol -> [State] -> [State]
    deltazero char st
        | st == [0] && char == '0' = [1]
        | st == [0] && char == '1' = [2]
        | st == [1] = [1]
        | st == [2] = [2]
        | otherwise = [-1]
```

2 NFA Implementation

This describes our implementation of Non-deterministic Finite State Automata. As in the definition, an NFA consists of a tuple $(Q, \Sigma, \delta, q_{start}, A)$ representing the set of states, the alphabet, the transition function, the start state and the accept states.

```
module NFA where

import Data.List
import Data.Maybe()
import DFA
data NFA = NFA { statesNF :: [State]
               , alphabetNF :: [Symbol]
               , deltaNF :: Symbol -> State -> [State]
               , startNF :: State
               , acceptstateNF :: [State] }
```

We implement a basic evaluation function that upon input from the string, evaluates if the string is in the language.

```
evaluateNF:: NFA -> String -> Bool
evaluateNF nf st = all ('elem' alphabetNF nf) st && -- captures elements of string in
    alphabet
    any ('elem' acceptstateNF nf) (stateArrNF' [startNF nf] st) where
        stateArrNF':: [State] -> String -> [State]
        stateArrNF' qs [] = qs
        stateArrNF' [] _ = []
        stateArrNF' [q] (x:xs) | q == -1 = []
                                | otherwise = stateArrNF' (deltaNF nf x q) xs --
                                                recursion on string
        stateArrNF' (q:qs) (x:xs) = stateArrNF' [q] (x : xs) 'union' stateArrNF' qs
                                (x : xs) --recursion on statespace
```

3 Implementation of ϵ -NFA-s

Here we modify our implementation of NFA-s to account for epsilon transitions. This requires some technical apparatus. First we give the type structure and define some helper functions to calculate ϵ -closure of states.

```
module ENFA where

import Data.List
import Data.Maybe()
import DFA
data ENFA = ENFA { statesENF :: [State]
                  , alphabetENF:: [Symbol]
                  , deltaENF :: Symbol -> State -> [State]
                  , epTrans:: [(State, State)]
                  , startENF:: State
                  , acceptstateENF:: [State]}

-- Taken from https://stackoverflow.com/questions/19212558/transitive-closure-from-a-list-
-- using-haskell
trClose :: Eq a => [(a, a)] -> [(a, a)]
trClose closure
    | closure == closureUntilNow = closure
    | otherwise                  = trClose closureUntilNow
    where closureUntilNow =
        nub $ closure ++ [(a, c) | (a, b) <- closure, (b', c) <- closure, b == b']

refClose:: Eq a => [a] -> [(a,a)] -> [(a,a)]
refClose as ps = nub (ps ++ [(x,x) | x <- as])

rtClose:: ENFA -> State -> [State]
rtClose nf q = [p | p <- statesENF nf , (q, p) 'elem' trClose rcl ] where
    rcl = refClose (statesENF nf) (epTrans nf)
```

We implement a basic evaluation function that upon input from the string, evaluates if the string is in the language.

```
unionL:: Eq a => [[a]] -> [a]
unionL = foldr union []

evaluateENF:: ENFA -> String -> Bool
evaluateENF nf st = all ('elem' alphabetENF nf) st && -- captures elements of string in
    alphabet
    any ('elem' acceptstateENF nf) (stateArrENF' [startENF nf] st) where
        stateArrENF':: [State] -> String -> [State]
        stateArrENF' qs [] = qs
        stateArrENF' [] _ = []
```

```

stateArrENF' [q] (x:xs) | q == -1 = []
                  | otherwise = stateArrENF' (unionL (map (deltaENF
                  nf x) (rtClose nf q))) xs
stateArrENF' (q:qs) (x:xs) = stateArrENF' [q] (x : xs) 'union' stateArrENF'
                  qs (x : xs) --recursion on statespace

```

4 Regular expressions

Here we implement regular expressions and a toolset to generate words that the regexp accepts.

```

module RegExp where

import System.Random
import DFA
import NFA()
import ENFA()

data RegExp = Epsilon | R [Symbol] | Union RegExp RegExp | Star RegExp | Con RegExp RegExp
            | Plus RegExp

instance Show RegExp where
  show Epsilon = "R e"
  show (R xs) = "R " ++ show xs
  show (Union r1 r2) = show r1 ++ " u " ++ show r2
  show (Star r) = "(" ++ show r ++ ")*"
  show (Con r1 r2) = show r1 ++ show r2
  show (Plus r) = "(" ++ show r ++ ")+

generateWord :: RegExp -> IO String
generateWord Epsilon = pure ""
generateWord (R xs) = pure xs
generateWord (Union r1 r2) = do
  i <- getStdRandom (randomR (0,1::Int))
  [generateWord r1, generateWord r2] !! i
generateWord (Con r1 r2) = do
  one <- generateWord r1
  two <- generateWord r2
  return (one ++ two)
generateWord (Star r) = do
  i <- getStdRandom (randomR (0,1::Int))
  [generateWord Epsilon, generateWord (Con r (Star r))] !! i
generateWord (Plus r) = generateWord (Con r (Star r))

```

A function to use for testing:

```

qc :: DFA -> RegExp -> Int -> IO ()
qc _ _ 0 = print "No counterexample found"
qc dfa ex n = do
  s <- generateWord ex
  if evaluate dfa s
  then qc dfa ex (n-1)
  else
    if s == ""
    then print "The DFA rejected the empty string"
    else print ("the DFA rejected " ++ s)

```

5 Transition functions

Here we implement several transition functions between objects. Powerset construction:

```
module Transitions where
import Data.List ( intersect, subsequences )
import NFA
import DFA
import ENFA

transNtoD:: NFA -> DFA
transNtoD (NFA sts alph del strt ac) =
  DFA (subsequences sts) alph del' [strt] [st | st <- subsequences sts, intersect st ac /=
    []] where
  del':: Symbol -> [State] -> [State]
  del' sy ls = unionL [del sy l | l <- ls]
```

Powerset construction:

```
transENTO:: ENFA -> DFA
transENTO (ENFA sts alph del eps strt ac) = let nf = ENFA sts alph del eps strt ac in
  DFA (subsequences sts) alph del' (rtClose nf strt) [st | st <- subsequences sts,
    intersect st ac /= []] where
  del':: Symbol -> [State] -> [State]
  del' sy ls = let nf = ENFA sts alph del eps strt ac in unionL (map (rtClose nf) lis)
    where
      lis = unionL [del sy l | l <- ls]
```

6 Wrapping it up in an executable

We will now use the library from Section 1 in a program.

```
module Main where

import DFA()
import Data.List()
import Data.Maybe()

main:: IO ()
main = undefined
```

The output of the program is something like this:

```
Hello!
[1,2,3,4,5,6,7,8,9,10]
[100,100,300,300,500,500,700,700,900,900]
[1,3,0,1,1,2,8,0,6,4]
[100,300,42,100,100,100,700,42,500,300]
GoodBye
```

Note that the above `showCode` block is only shown, but it gets ignored by the Haskell compiler.

7 Simple Tests

We now use the library QuickCheck to randomly generate input for our functions and test some properties.

```
module Main where

import Basics

import Test.Hspec
import Test.QuickCheck
```

The following uses the HSpec library to define different tests. Note that the first test is a specific test with fixed inputs. The second and third test use QuickCheck.

```
main :: IO ()
main = hspec $ do
  describe "Basics" $ do
    it "somenumbers should be the same as [1..10]" $
      somenumbers 'shouldBe' [1..10]
    it "funnyfunction: result is within [1..100]" $
      property (\n -> funnyfunction n 'elem' [1..100])
    it "myreverse: using it twice gives back the same list" $
      property $ \str -> myreverse (myreverse str) == (str::String)
```

To run the tests, use `stack test`.

To also find out which part of your program is actually used for these tests, run `stack clean && stack test`. Then look for “The coverage report for ... is available athtml” and open this file in your browser. See also: https://wiki.haskell.org/Haskell_program_coverage.

8 Conclusion

Finally, we can see that [LW13] is a nice paper.

References

- [LW13] Fenrong Liu and Yanjing Wang. Reasoning about agent types and the hardest logic puzzle ever. *Minds and Machines*, 23(1):123–161, 2013.